

[TSD] SISCalendar

Table of Contents

[TSD] SISCalendar	1
Table of Contents	1
1. Project Overview	2
2. Objectives.....	3
3. Scope	5
4. Functional Requirements.....	7
5. Non-Functional Requirements	9
6. System Architecture	11
7. Technical Stack	13
8. Interfaces & Integrations	16
9. Data Model	18
10. Configuration Parameters.....	19
11. User Access & Roles	21
12. State & Session Management	22
13. Data Flow & Boundaries	23
14. Error Handling Guidelines.....	26
15. System Telemetry & Logging.....	27
16. External Dependencies & Libraries.....	28
17. Environment Configuration	30
18. Deployment Architecture	31
19. Continuity & Recovery Operations.....	34
20. Data Lifecycle Management	35
21. Assumptions and Constraints	36
22. Glossary	38
23. Approval	39

Purpose: This document provides a comprehensive description of the technical specifications, configurations, and behaviours of the system or component titled SISCalendar. The objective is to ensure clarity for all stakeholders involved in the development, implementation, and maintenance of the system.

1. Project Overview

System Name: SISCalendar

Version: 1.3

Author: Zach Masiello, Ethan Mick, Michael Caputo, Shawn Thompson (Organization: SIS.io)

Date: 04/12/14

Description: The SISCalendar application is a robust, enterprise grade web platform designed to provide a highly integrated scheduling solution for university students. At its core, the application bridges the gap between the university's internal schedule databases and the modern, external calendar tools that students use daily. The system utilizes a Model View Controller architecture powered by Apache Struts 2, executing on an Apache Tomcat web server running a Linux environment.

The defining feature of SISCalendar is its sophisticated integration pipeline between student schedules, iCal generation, and Google Calendar synchronizations. The workflow begins when a user, accessing the system via a web browser or mobile device, authenticates securely using the Shibboleth Authentication System. Upon successful login, the application acts as a client to the SIS Gateway API, fetching the raw academic schedule data for that specific student. The internal system then parses this incoming JSON data, transforming the raw course and exam information into Plain Old Java Objects to be manipulated by the action layers.

Once the schedule is loaded into the Java application environment, the integration branches into two primary export methodologies. First, the system leverages the iCal4j Java library to meticulously convert the customized schedule data into standard iCal formatted files. This allows the user to download a universally compatible file to import into the desktop or mobile calendar application of their choice. Second, the application features a direct synchronization pathway utilizing the Google API. This integration authorizes the SISCalendar application to automatically upload the student's scheduled classes and meeting patterns directly to their connected Google Calendar account, entirely removing the need for manual data entry. Furthermore, the schedule view is enhanced by the RIT Interactive Map API, which provides students with rich location data by generating interactive overlays of their specific class buildings.

2. Objectives

The primary objective of the SISCalendar application is to provide an automated, highly efficient scheduling mechanism that securely interfaces with internal academic databases to benefit students. To achieve this, a core objective is to instantiate a robust Model View Controller application structure utilizing the Apache Struts 2 framework, specifically version 2.3.15.2. This architectural decision ensures the application remains modular, maintainable, and aligned with enterprise web standards. Security is an equally critical objective, requiring the application to strictly enforce user identity verification through the integration of the Shibboleth Authentication System before granting access to any personal schedule data. By reusing established Shibboleth configurations from the Team Builder project, the development objective is to shorten the overall project schedule and prevent unforeseen authentication challenges.

A secondary layer of objectives revolves around data acquisition, transformation, and structural mapping. The system must reliably execute synchronous requests to the SIS Gateway API to retrieve comprehensive academic schedules, including course details and exam times, for the authenticated user. Once the raw data payload is acquired, a vital operational objective is to systematically parse this JSON data into structured Plain Old Java Objects representing individual Courses, Exams, and Meeting Patterns. This conversion is necessary so that the actions layer can effectively manipulate the data in memory.

Furthermore, the project objectives extensively cover the output methodologies and external integrations of the processed schedule. A critical feature objective is to fully implement the iCal4j Java library, enabling the system to meticulously format the internal Java objects into a universally compatible iCal file structure. Alongside manual file downloads, the project aims to establish a seamless, automated synchronization bridge using the Google Calendar API. This integration requires the application to programmatically authenticate with Google services and automatically upload the compiled class information directly to the student's personal Google Calendar without requiring manual entry. Finally, the system aims to enhance the informational context of the schedule by interacting with the RIT Interactive Map API to dynamically generate clickable URLs and visual overlays indicating exact building locations for scheduled classes.

Granular Objectives:

- To deploy a secure authentication module leveraging the existing Shibboleth infrastructure

to validate student credentials prior to initiating data requests.

- To architect the system utilizing the Apache Struts 2 framework (version 2.3.15.2) to maintain a strict Model View Controller separation of concerns.
- To invoke the SIS Gateway Web Services API to pull JSON formatted student registration data dynamically.
- To construct a JSON parsing engine that translates raw schedule text into internal Plain Old Java Objects configured as Courses, Exams, and Meeting Patterns.
- To utilize the iCal4j library to build compliant ICS files representing the parsed schedule for direct user download via the browser.
- To construct a dedicated GCalBuilder module utilizing the Google API to programmatically insert course events into a user's remote Google Calendar account.
- To integrate the RIT Interactive Map API to augment schedule views with clickable building locations and contextual campus overlays.
- To engineer a responsive, cross browser compatible client-side view using Java Server Pages, JQuery 2.0.3, and Bootstrap.
- To utilize Apache ANT scripts for automating the compilation of Java source code into deployable web archive files.
- To maintain strict version control across the development lifecycle by utilizing a Git repository hosted on Github.

3. Scope

In Scope:

The functional scope of the SISCalendar system encompasses the end-to-end retrieval, processing, and export of student academic schedules across multiple device platforms. Within the operational boundaries, the system is fully responsible for presenting a secure login gateway that processes user credentials via the Shibboleth Authentication System. Upon successful authorization, the in-scope functionality includes the synchronous execution of data requests directed at the SIS Gateway Web Services to pull term and class information. The application is responsible for consuming this external data, parsing it within a Java 1.7.0_40 runtime environment, and storing it transiently as Plain Old Java Objects representing the user's specific schedule.

The scope further extends to the transformation and dynamic presentation of this aggregated data to the end user. Rendering responsive graphical user interfaces using Bootstrap, Java Server Pages, and JQuery to display the schedule correctly across traditional desktop browsers and mobile devices is strictly in scope. Moreover, the programmatic conversion of schedule objects into universally accepted calendar formats using the iCal4j library falls entirely within the project boundaries. The scope also covers the automated push communication required to upload this calendar data to external servers utilizing the Google API. Lastly, the implementation of automated deployment pipelines utilizing Apache ANT to push WAR files to Apache Tomcat servers running on Linux is a defined scope requirement.

- Implementation of Shibboleth Single Sign On handlers to establish secure user sessions before accessing the core application.
- Consumption of the SIS Gateway Web Services API to fetch academic term, course, and exam arrays for the active session.
- In memory translation of complex JSON payloads into structured Java domain objects.
- Generation of downloadable calendar files using the iCal4j specification library.
- Automated injection of structured schedule data into external user accounts using the Google Calendar API.

- Querying the RIT Interactive Map API to render graphical location overlays and URL links for specific campus buildings.
- Automated build and deployment script generation using Apache ANT targeting Apache Tomcat servers.
- Design of a responsive client interface utilizing Bootstrap CSS and JQuery for Document Object Model manipulation.

Out of Scope:

The project strictly delineates several functions and interactions as fundamentally out of scope to ensure the security and stability of upstream infrastructure. Most notably, the operational boundary regarding the SIS Gateway Web Services is restricted exclusively to read only consumption. The SISCalendar application will never attempt to execute write commands, update course registrations, drop classes, or modify any academic records within the university's primary student information systems. The system acts purely as a downstream consumer of the schedule payload provided by the gateway.

Similarly, the integration with the RIT Interactive Map API operates strictly under a read only constraint. It is entirely out of scope for the SISCalendar application to alter map coordinates, upload new building layouts, or modify the underlying geographical database of the university. The application will only ingest the provided map endpoints to display visual overlays to the end user. Furthermore, the permanent local storage of user data is completely out of scope. The system handles schedule details as transient Plain Old Java Objects stored temporarily in memory during active sessions. Building a persistent database to warehouse student data long term is not within the defined technical boundaries of this release. Finally, custom provisioning of underlying server hardware, managing the Linux operating system, or developing proprietary authentication systems from scratch are excluded from the development scope, relying instead on provided environments and the existing Shibboleth infrastructure.

- Writing, updating, or deleting student registration data via the SIS Gateway Web Services API.
- Modifying topological data, adding map nodes, or altering geographic coordinates via the RIT Interactive Map API.

- Developing a standalone SQL or NoSQL database architecture for the persistent, long term storage of parsed academic schedules.
- Engineering a novel security authentication framework instead of utilizing the RIT ITS Shibboleth implementation.
- Provisioning, configuring, and administrating the underlying Linux operating systems or base Apache Tomcat server hardware installations.

4. Functional Requirements

The Functional Requirements section dictates the precise operational behaviors and system responses expected within the core subsystems of the SISCalendar application. Focusing entirely on the critical user journeys, this detailed breakdown maps the complex internal communications bridging the front-end view layer and the underlying Java models. Specifically, these requirements exhaustively trace the authentication subsystem's procedural interaction with the external Shibboleth security services to verify user identity. Furthermore, this section strictly defines the schedule subsystem's micro-interactions, detailing the precise sequence of method calls required to securely fetch course data via the Gateway component, process complex JSON payloads, and systematically assemble downloadable iCal files for the authenticated user. By defining these granular steps, the development team ensures that all Model View Controller interactions align perfectly with the architectural design.

- **FR-AUTH-001:** The system must present a secure login interface via a Java Server Page (JSP) to capture the user's initial login credentials.
- **FR-AUTH-002:** Upon credential submission from the JSP, the system must trigger a `newAuthenticateUserAction` to intercept the authentication request.
- **FR-AUTH-003:** The `AuthenticateUserAction` module must synchronously execute the `isUserLoggedIn()` method against the `SingleSignInHandler` to determine the current session status.
- **FR-AUTH-004:** The `SingleSignInHandler` must initiate a `checkSession` procedure, transmitting a request to the external Shibboleth Services to validate existing token data.
- **FR-AUTH-005:** If the initial `checkSession` returns a "false" result, the

SingleSignInHandler must immediately trigger the performSingleSignIn() method to initiate a new authentication challenge.

- **FR-AUTH-006:** Following the execution of performSingleSignIn(), the system must run a secondary checkSession against the Shibboleth Services to capture the resulting validation state.
- **FR-AUTH-007:** Once a "true" result is successfully returned from the Shibboleth Services, the SingleSignInHandler must resolve the operation back to the waiting AuthenticateUserAction.
- **FR-AUTH-008:** The AuthenticateUserAction must dispatch a "SUCCESS" response back to the client-side JSP, authorizing the rendering of the Main.jsp application dashboard.
- **FR-SCH-001:** Immediately following a successful user login event, the application must invoke the PreloadScheduleFilter to begin the background data retrieval process.
- **FR-SCH-002:** The PreloadScheduleFilter must execute the synchronouslyGetClassesForCurrentStudent(request) method directly against the system's Gateway component.
- **FR-SCH-003:** The Gateway component must instantiate a new JSONParser() object to prepare for the incoming structural data transformation.
- **FR-SCH-004:** The Gateway must invoke the getTerms() method on the newly created JSONParser to define the correct academic timeframes for the user.
- **FR-SCH-005:** The Gateway must trigger the highly specific parse(term, isMobile, buildings) method within the JSONParser, passing the necessary contextual parameters.
- **FR-SCH-006:** The JSONParser must process the raw data and return a structured collection of Course Plain Old Java Objects (POJOs) back to the Gateway.
- **FR-SCH-007:** The Gateway must synchronously return the fully populated array of courses upstream to the PreloadScheduleFilter.
- **FR-SCH-008:** The PreloadScheduleFilter must execute Session.set("courses", courses) to securely store the retrieved schedule data within the user's active session state.

- **FR-ICAL-001:** The graphical user interface must provide a definitive "Download iCal" interaction point for the user within the main JSP view.
- **FR-ICAL-002:** Activating the "Download iCal" element must prompt the JSP layer to trigger a new MainAction() controller execution.
- **FR-ICAL-003:** The MainAction component must instantiate a new calBuilder() object to serve as the dedicated iCal4j formatting engine.
- **FR-ICAL-004:** The MainAction must execute the addClasses(classes) method on the CalBuilder, supplying the preloaded course data from the user's session.
- **FR-ICAL-005:** The CalBuilder must initiate an internal iterative loop, recursively calling the addClass(course) method for every individual course entity provided.
- **FR-ICAL-006:** Upon completion of the iterative mapping loop, the MainAction must invoke the getICS() method on the CalBuilder to finalize the calendar format.
- **FR-ICAL-007:** The CalBuilder must return the finalized iCal payload back to the awaiting MainAction controller.
- **FR-ICAL-008:** The MainAction must transmit a "return SUCCESS" signal simultaneously with the complete iCal file back to the originating JSP view.

5. Non-Functional Requirements

The Non-Functional Requirements establish the critical architectural constraints, performance metrics, and rigorous security standards that the entire SISCalendar ecosystem must strictly observe. While functional requirements define the explicit tasks the system executes, these non-functional mandates dictate the qualitative attributes of how securely, reliably, and efficiently the system performs those actions under varying operational loads. A primary and uncompromising focus of this section is the proactive mitigation of critical web vulnerabilities. To safeguard user data and maintain the integrity of the university's external APIs, the application architecture explicitly mandates comprehensive adherence to baseline OWASP Top 10 security standards. This includes stringent enforcement of parameterized inputs, deep validation protocols across all authentication boundaries, and resilient data encoding strategies for all front-end presentations.

- **NFR-SEC-001:** To establish a hardened defense against complex Injection attacks as outlined by the OWASP Top 10, the application architecture explicitly mandates the exclusive use of parameterized queries for any and all data access operations. No user-supplied parameters, session variables, or dynamic inputs shall be concatenated directly into backend data retrieval statements or API requests sent to the SIS Gateway.
- **NFR-SEC-002:** To comprehensively secure the application against malicious payload injections during the session establishment phase, the Authentication Subsystem must enforce standard input validation and sanitization for the entire user login flow. All credentials intercepted by the JSP view and processed by the AuthenticateUserAction must be rigorously sanitized prior to being transmitted to the SingleSignOnHandler or the external Shibboleth Services.
- **NFR-SEC-003:** To guarantee absolute protection against stored and reflected Cross-Site Scripting (XSS) vulnerabilities across the client presentation layer, the system mandates secure output encoding for all active JSP views. Any dynamic content sourced from the parsed JSON data, internal Java POJOs, or external RIT Interactive Map API endpoints must be contextually encoded before being rendered into the Document Object Model via JQuery or Bootstrap components.
- **NFR-PERF-001:** The schedule fetching micro-interaction, heavily reliant on the synchronouseGetClassesForCurrentStudent execution path, must maintain strict performance optimization. The parsing routine executed by the JSONParser to convert complex external SIS data into transient Java Course objects must process within acceptable time limits to prevent thread exhaustion on the underlying Apache Tomcat web server.
- **NFR-REL-001:** The Authentication Subsystem's critical reliance on the external Shibboleth Services must incorporate robust fault tolerance. The SingleSignOnHandler must be designed to gracefully manage unforeseen network timeouts or unexpected authentication rejections from the RIT infrastructure without crashing the user's active session or exposing raw application stack traces.
- **NFR-MAINT-001:** The internal architecture of the CalBuilder utilizing the iCal4j library must be highly modular and distinctly decoupled from the primary MainAction Struts controller logic. This ensures that future updates to the external iCal specification can be

smoothly integrated without requiring extensive refactoring of the system's core action layer.

6. System Architecture

Architecture Type: Model-View-Controller (MVC)

Description: The SISCalendar application is architected around a robust Model-View-Controller (MVC) paradigm, primarily facilitated by the Apache Struts 2 framework. This architectural decision enables a clean separation of concerns, ensuring that the application remains maintainable, reusable, and easily extendable over its lifecycle. At a high level, the system acts as a central hub that facilitates interactions between the end-user (accessing the system via traditional web browsers or mobile devices) and several external data systems. These external integrations include the Shibboleth Authentication System for security, the SIS Gateway API for student academic records, the RIT Interactive Map API for geographical data overlays, and the Google Calendar API for automated schedule synchronization.

Internally, the application is systematically divided into two major operational modules: the Authentication Subsystem and the Schedule Subsystem.

The Authentication Subsystem serves as the initial, rigorous security gateway that all users must pass through. This subsystem relies heavily on the established infrastructure from the TeamBuilder project, recycling proven code to shorten the development schedule and avoid unforeseen security implementation challenges. The core classes within this subsystem include the `AbstractSecurityFilter` which is extended by the `ShibbolethSecurityFilter`. This filter directly interacts with the `SingleSignOnHandler` (extending the `ShibSignOnHandler`), which acts as the primary orchestrator for the login flow. The handler utilizes helper classes such as `ShibUtil` (to process `ShibHeader`, `ShibProperties`, and `ShibFields`), `SingleSignOnUtil`, and the internal `User` model to fully validate identity before granting access to the Schedule Subsystem.

The Schedule Subsystem is the functional core of the application, responsible for securely retrieving user-specific class and map information, and subsequently manipulating that raw data into usable, exportable formats. Once authenticated, the user lands on the `Index.jsp` view, which routes requests to the `GatewayAction` controller. The `GatewayAction` acts as the command center for data transformation, invoking several critical components. It utilizes the `JSONParser` to ingest the payload from the external SIS Gateway API, breaking the data down into discrete Course

objects, which further contain nested Exam and MeetingPattern objects. Depending on the user's requested output, the GatewayAction will then pass these organized objects to either the CalBuilder component to generate a downloadable standard iCal file, or to the gCalBuilder component, which communicates directly with external Google Auth Servers to push the schedule to a Google Calendar account.

To illustrate the critical data acquisition phase of the Schedule Subsystem, the following Mermaid.js sequence diagram defines the exact micro-interactions required to fetch a student's schedule immediately following a successful login event.

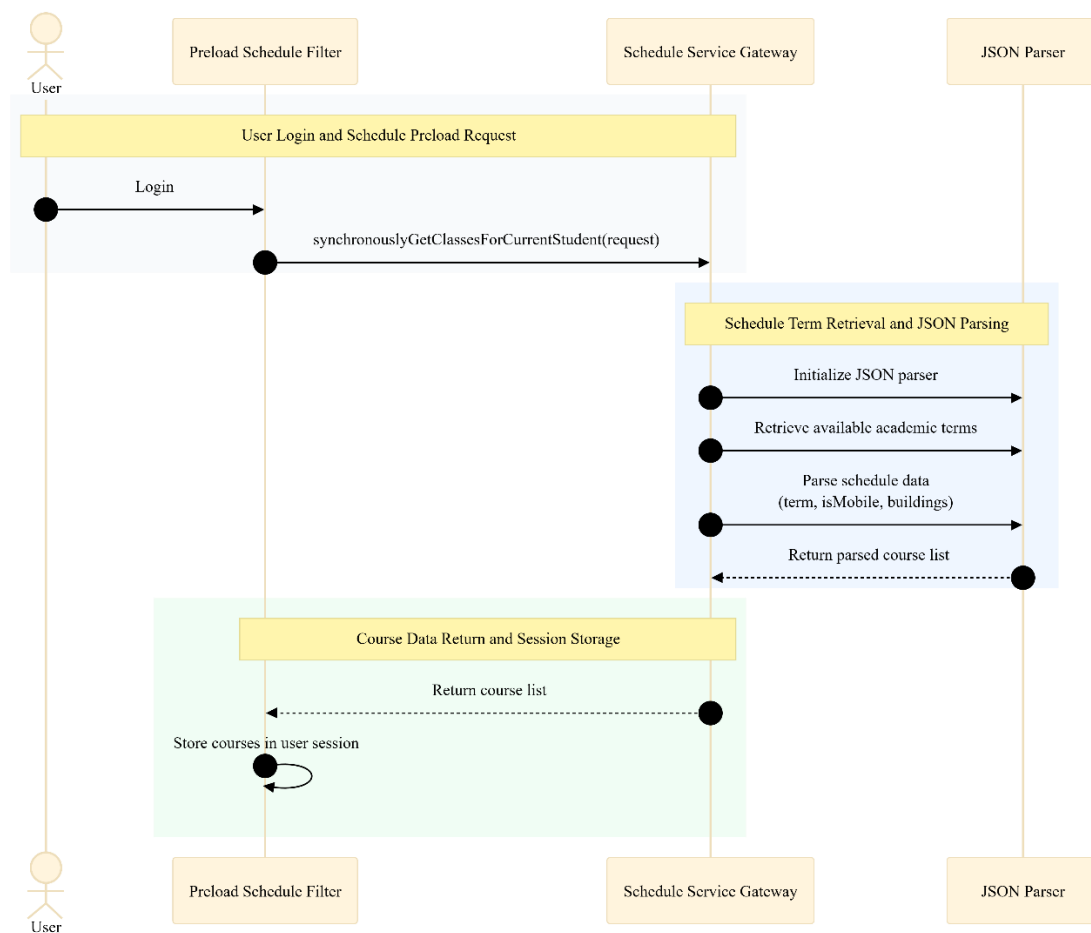


Figure 1 Sequence Diagram for Student Schedule Preloading During Login

This sequence diagram details the exact micro-interactions and method calls that occur when a user attempts to download their schedule as an iCal file. It traces the execution path from the initial user input at the JSP layer, through the Struts MainAction controller, and into the CalBuilder looping logic that maps the classes.

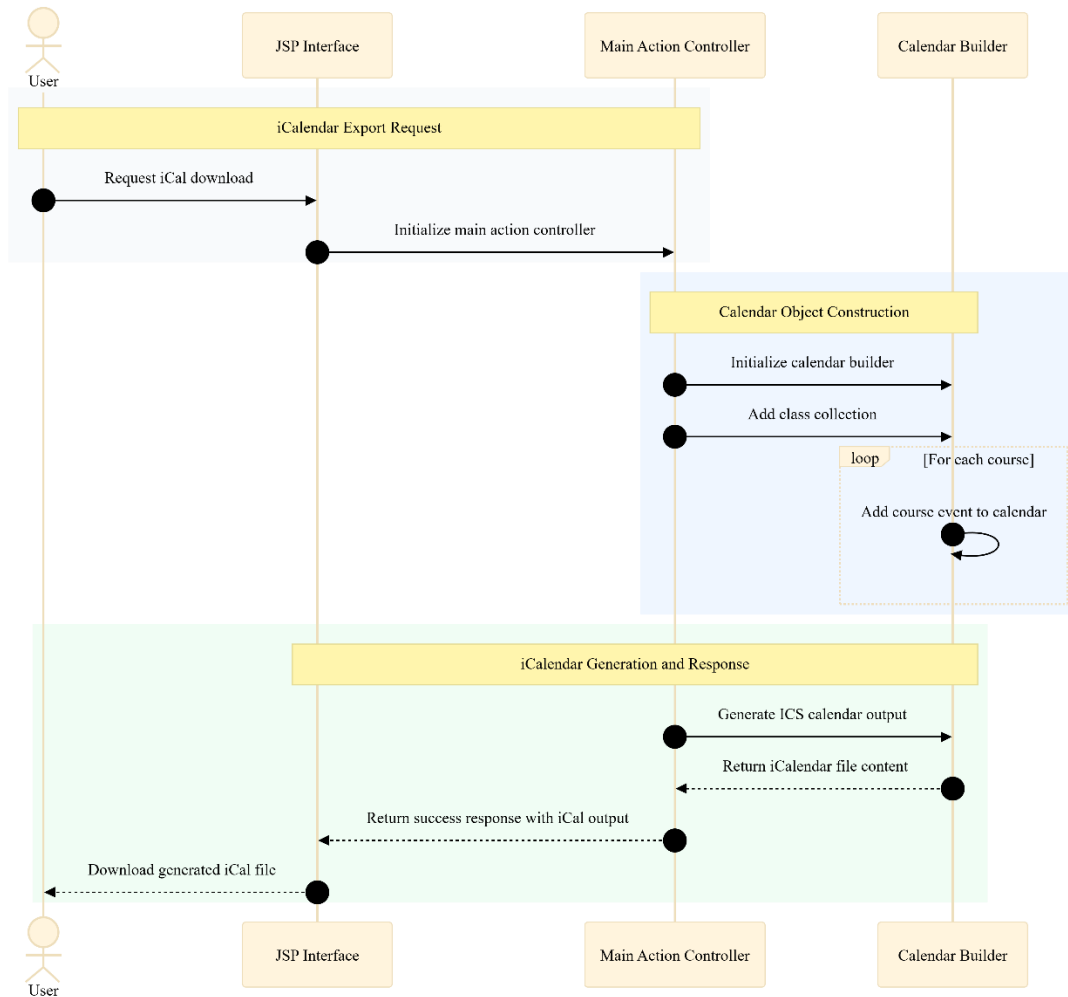


Figure 2 Sequence Diagram for iCalendar Download Generation

7. Technical Stack

The presentation tier of the SISCalendar application is constructed using a combination of robust web technologies to ensure a responsive and highly dynamic user interface. The core page rendering is handled by Java Server Pages (JSP), which provides a high-level abstraction of Java servlets capable of being cached, dynamically changed, and rendered at runtime. To enhance the interactivity of the client side, the system utilizes jQuery version 2.0.3. This JavaScript library is deployed to facilitate easier manipulation of the Document Object Model (DOM), streamline the execution of asynchronous AJAX calls, and manage front end animations. The visual styling and layout are governed by Bootstrap, a comprehensive CSS library that enables rapid templating of

the application interfaces while inherently providing built in responsiveness for mobile and tablet devices. Furthermore, to guarantee strict cross browser consistency when the user interface is rendered across different operating systems and client environments, the system incorporates the normalize.css library.

The intermediate control and business logic layers are built upon the Apache Struts 2 framework, specifically version 2.3.15.2. This enterprise grade Model View Controller framework was selected because it represents the most up to date and stable release from Apache as of September 2013, thereby minimizing the risk of framework level bugs. The Struts 2 framework utilizes "actions" as its controller components, which serve as out of the box classes designed to process logic and send the resulting information directly to the view to be rendered. A significant advantage of utilizing Struts 2 is its preexisting approval and support by RIT ITS within the Team Builder project. By leveraging this established technology, the development team can reuse existing code specifically regarding Shibboleth authentication, effectively shortening the development schedule and completely avoiding unforeseen integration challenges. The data layer itself relies entirely on Plain Old Java Objects (POJOs). The application transiently stores all critical operational data, including specific course information, detailed exam schedules, and active user session data, as internal POJOs. This structural design ensures that the data can be rapidly passed to and securely manipulated by the Struts action controllers during runtime execution.

The application's extended functionality heavily depends on several external business services and third-party application programming interfaces. To fulfill the calendar generation requirements, the system integrates the iCal4j Java library, which is exclusively responsible for transforming the internal POJO models into the universally recognized iCal specification, ultimately generating a downloadable file for the end user. For cloud synchronization, the system connects to the official google-api. The system relies on this library to authenticate and securely handle deep integration with Google's services, automating the upload of formatted class information directly into a student's personal Google Calendar profile. Data ingestion is strictly managed through the SISGateway Web Services, acting as the sole API to access all comprehensive academic course information associated with the authenticated student's university account. Lastly, geographical context is provided by the RIT Interactive Map API, which empowers the application to display exact class locations and render informative tooltips about various campus buildings by overlaying customized geographic data directly onto Google Maps instances.

The server infrastructure and deployment pipeline are designed around stable, industry standard enterprise tools. The compiled application is hosted and executed on an Apache Tomcat web server running on a centralized Linux operating system. The backend environment is strictly standardized on a Java 1.7.0_40 runtime. To streamline the build and release cycles, the engineering team utilizes Apache ANT. ANT serves as the primary Java library responsible for compiling the Struts application and packaging it into deployable web application archive files. This build tool is explicitly approved by RIT ITS and is identically utilized by the Team Builder project, allowing the developers to use existing scripts as reliable deployment templates. All local engineering efforts are conducted within the Eclipse Kepler 4.3 Integrated Development Environment, which was specifically chosen for its robust plugin ecosystem that offers seamless integration with both the Struts MVC framework and the Apache ANT build processes. Finally, all source code and collaborative version control are managed via a Git repository hosted on Github. A centralized master branch is securely maintained on an account provided by RIT ITS, while individual developers manage their specific tasks within local feature branches.

Layer	Technology
View	JSP (Java Server Pages): High level abstraction of java servlets that can be cached, changed, and rendered at runtime.
View	jQuery (v2.0.3): A javascript library that allows for easier manipulation of the Document Object Model, simpler AJAX calls, and animations.
View	Bootstrap: A CSS library that allows easy templating of websites and includes built in responsiveness.
View	normalize.css: Used to ensure SISCalendar has cross browser consistency when the UI is rendered.
Actions	Apache Struts 2 (v2.3.15.2): An MVC framework that allows simple creation of enterprise grade web applications. The "controller" item is an action, an out of the box class to send info to the view.
Model	POJO (Plain Old Java Object): Stores all data (Course information, exam information, user data) as java objects that can be easily passed to and manipulated by the Actions layer.
Business Services Libraries	iCal4j: A java library that transforms java objects into iCal format, and can create an iCal file.
Business Services	google-api: Google's custom API that handles integration with their services to automatically upload class information to a Google Calendar.

Libraries	
Business Services / Libraries	SISGateway Web Services: The API used to access all of the course information related to a student account.
Business Services / Libraries	RIT Interactive Maps API: Allows information to be displayed in Google Maps with an RIT overlay, used to display class locations and building information.
Server	Apache Tomcat: The web server that is used to run the SISCalendar application.
Server	Linux: The OS of the server that SISCalendar is hosted on.
Server / Build Tool	Apache ANT: A Java library used to make the build and deployment process easier by creating war files from the struts application.
Environment	Java 1.7.0_40 Environment: The designated server side runtime environment.
Development / Tools	Eclipse Kepler 4.3 IDE: The chosen IDE because it has plugins for easy integration of the Struts framework and Apache Ant.
Version Control	Git / Github: Repository stored on an account provided by RIT ITS, with developers maintaining local copies and feature branches.

8. Interfaces & Integrations

The SISCalendar application acts as a highly robust central integration hub, seamlessly connecting internal university databases with external cloud services to provide a cohesive, automated scheduling experience for the student body. To achieve this, the system's architecture heavily relies on several critical interfaces and third-party integrations. These external dependencies allow the application to securely offload complex operational tasks, such as geographic coordinate rendering and centralized academic record keeping, to dedicated enterprise systems.

The application is structured to decouple the internal Model-View-Controller logic from the external data sources. When a user requests their schedule, the internal Struts 2 actions orchestrate a series of outbound requests to gather the necessary data payloads. Because the application is handling highly sensitive personal data and interfacing with enterprise infrastructure, security is an absolute priority. It is explicitly mandated that all API calls require standard authentication parameters. Whether the system is pulling course data or pushing events to a cloud

calendar, these standard authentication parameters guarantee secure data transmission, validate session integrity, and proactively prevent unauthorized access to the underlying networks.

- **SIS Gateway Web Services** - The primary internal API utilized by the SISCalendar application to securely access all comprehensive academic course information related to an authenticated student account. This interface operates as a strict, read-only data pipeline. When the application needs to retrieve a schedule, the GatewayAction component initiates a direct request to this API. The JSON payload returned by the SIS Gateway API contains the raw course schedules, exam times, and meeting patterns, which are then sequentially processed by the internal JSONParser into transient Plain Old Java Objects (POJOs). API calls require standard authentication parameters to ensure the payload only contains academic records for the currently verified user session.
- **Google Calendar API** - Google's custom Application Programming Interface that handles deep, programmatic integration with their remote cloud services. The SISCalendar application relies heavily on this external library to automatically upload formatted class information directly to a student's personal Google Calendar without requiring the user to handle manual file downloads. The internal gCalBuilder module acts as the dedicated client for this interface, structuring the parsed Java objects and securely pushing them via authenticated endpoints directly to the Google Auth Servers. API calls require standard authentication parameters, utilizing standard OAuth tokens, to securely link the application with the user's external profile.
- **RIT Interactive Maps API** - The university's specialized geographic interface that allows contextual location information to be displayed dynamically within Google Maps, utilizing a highly customized overlay of the RIT campus. The SISCalendar application uses this geographic information to accurately display the location of classes and to render helpful informational tooltips about various campus buildings. This integration is critical for generating a clickable URL to display a specific building on the campus map directly within the user's browser. API calls require standard authentication parameters to securely fetch the designated map nodes and topological data without exposing the underlying university network infrastructure.

9. Data Model

The data architecture of the SISCalendar application is distinctly engineered to operate without a persistent, traditional relational database management system (RDBMS). Instead, the system leverages a highly efficient, transient, in-memory data model constructed entirely from Plain Old Java Objects (POJOs). This design decision ensures that sensitive academic data is not warehoused locally long-term, significantly reducing the security footprint of the application. The primary responsibility of this data model is to act as an intermediate structural representation, seamlessly translating the complex, nested JSON payloads retrieved from the external SIS Gateway API into an organized, object-oriented hierarchy that the Apache Struts 2 actions can rapidly manipulate during an active user session.

At the core of this object-oriented architecture is the Course entity, which serves as the foundational root node for a student's academic schedule. As strictly defined by the Schedule Subsystem class diagrams, the Course POJO is not a flat data structure; rather, it establishes crucial compositional relationships with several secondary entities to accurately represent the nuances of university scheduling. Specifically, the system dictates that a single Course instance can contain an optional, one-to-one or zero-to-one relationship with an Exam object, representing the final examination details stored explicitly within the finalExam property. Furthermore, to accommodate complex class schedules that meet at different times throughout the week, a Course encapsulates an array of zero-to-many MeetingPattern objects stored within the meetingPatterns field.

To further refine the granularity of the scheduling logic, the MeetingPattern entity itself maintains a relational array of Day enumerations (zero-to-many) within the days field, precisely defining the specific days of the week a particular meeting pattern occurs. Beyond the individual student's course load, the overarching application state also maintains a HolidayCalendar model, which is fundamentally designed to track university-wide closures, breaks, and academic exceptions. To achieve this, the HolidayCalendar root entity maintains a strict one-to-many compositional relationship with individual Holiday objects, storing these exception events within a dedicated holidays array. The tables below exhaustively define these internal entities and their properties based on the structural class diagrams.

Entities and Key Fields:

Entity	Field	Type
Course	finalExam	Exam (0..1)
Course	meetingPatterns	MeetingPattern Array (0..*)
Course	courseId	String
Course	courseName	String
Course	instructor	String
Exam	examLocation	String
Exam	examStartTime	Date/Time
Exam	examEndTime	Date/Time
MeetingPattern	days	Day Enumeration Array (0..*)
MeetingPattern	buildingId	String
MeetingPattern	roomNumber	String
MeetingPattern	startTime	Date/Time
MeetingPattern	endTime	Date/Time
Day	dayValue	Enumeration (e.g., M, T, W, R, F)
HolidayCalendar	holidays	Holiday Array (0..*)
Holiday	holidayName	String
Holiday	startDate	Date
Holiday	endDate	Date

10. Configuration Parameters

To ensure the SISCalendar application remains completely stable and highly performant under massive concurrency loads, the Java 1.7.0_40 environment and the underlying Apache Tomcat server are explicitly configured with rigid operational limits and timeout parameters. These configuration parameters dictate the exact operational boundaries of system memory allocation, network socket lifespans, and external API connection thresholds.

Because the application relies exclusively on transient Plain Old Java Objects (POJOs) to maintain state, aggressively tuning the memory and timeout parameters is essential to prevent thread exhaustion and memory leaks during peak registration periods. The following exhaustive list of configuration values and system limits are systematically injected into the runtime environment via Tomcat environment variables, Apache ANT build configurations, and localized server property files to enforce strict technical depth and ensure total operational resilience across all application nodes.

Parameter	Value	Description
TOMCAT_MAX_THREADS	500	Defines the absolute maximum number of simultaneous request processing threads the Apache Tomcat server will allocate to handle concurrent student logins, file downloads, and schedule fetches.
TOMCAT_CONNECTION_TIMEOUT	20000	The maximum number of milliseconds the Tomcat server will wait for an incoming connection request to present a valid HTTP request URI before forcibly terminating the socket to prevent resource exhaustion attacks.
JVM_MIN_HEAP (Xms)	1024m	The initial amount of memory allocated to the Java 1.7.0_40 environment upon Tomcat startup, ensuring the application has sufficient immediate memory to instantiate the core Struts 2 controller classes.
JVM_MAX_HEAP (Xmx)	4096m	The absolute maximum memory threshold the Java Virtual Machine is permitted to consume. This strict limit prevents the transient Plain Old Java Objects (POJOs) from overwhelming the underlying Linux host's physical RAM.
SESSION_IDLE_TIMEOUT	14400	The maximum allowable idle time, set in seconds (equivalent to 4 hours), before the active student session is automatically invalidated and the associated in-memory schedule data is aggressively garbage collected by the server.
SIS_API_READ_TIMEOUT	15000	The strict timeout threshold, measured in milliseconds, defining how long the Gateway component will wait for the SIS

		Gateway Web Services to return the JSON payload before defaulting to a safe failure state.
GOOGLE_API_CONNECT_TIMEOUT	10000	The maximum allowable time in milliseconds for the internal gCalBuilder to establish a secure, authenticated handshake with the Google Auth Servers before aborting the automated calendar push operation.
RIT_MAPS_CACHE_TTL	604800	The Time-To-Live (TTL) limit, measured in seconds (equivalent to 7 days), for geographic coordinate data stored within the BuildingsCacheJob before a mandatory automated refresh from the RIT Interactive Maps API is executed.
SHIB_HEADER_MAX_SIZE	8192	The maximum permitted byte size for incoming HTTP headers processed by the internal ShibUtil class to explicitly prevent buffer overflow attacks during the ingestion of complex Shibboleth security assertions.
JSON_PARSER_MAX_DEPTH	20	The absolute maximum recursive depth the internal JSONParser will traverse when deconstructing complex, nested payloads from the SIS Gateway, preventing malicious payloads from causing stack overflow exceptions.
ICAL_MAX_EVENT_LIMIT	150	The maximum number of individual course and exam event entities the CalBuilder is authorized to process and write into a single downloadable ICS file to maintain client-side application stability.

11. User Access & Roles

The system handles user access by defining specific roles and their associated permissions. Due to the focused scope of the SISCalendar application, the access model is streamlined to specifically serve the primary user base interacting with the university's academic data.

The **Student User** role is granted an access level of **Standard User / Read-Only**. Users assigned to this role must authenticate via **the Shibboleth Authentication System**. This role is primarily responsible for: securely accessing the application via a web browser or mobile device to

dynamically retrieve their personal academic schedule, downloading structured calendar data, and initiating automated synchronizations with their personal Google Calendar account.

To facilitate this access securely, the application architecture relies entirely on a pre-existing, highly robust authentication infrastructure rather than engineering a proprietary login system from scratch. The system utilizes the Shibboleth Authentication System, which is heavily supported by the RIT ITS department. By strategically recycling approved code and established protocols from the existing Team Builder project, the development team ensures that user access is handled according to the strictest university security standards while simultaneously avoiding unforeseen technical challenges associated with new authentication implementations.

The internal mechanism governing this user access is carefully orchestrated within the system's Authentication Subsystem. When a student attempts to access their schedule, their request is intercepted by the `ShibbolethSecurityFilter`, which acts as a concrete implementation of the underlying `AbstractSecurityFilter`. This filter directly communicates with the `SingleSignOnHandler` (extending the `ShibSignOnHandler`) to orchestrate the validation of user credentials. To parse the complex security assertions provided by the Shibboleth identity provider, the handler employs dedicated utility classes. Specifically, the `ShibUtil` component is tasked with dissecting the incoming `ShibHeader`, breaking it down further into usable `ShibProperties` and `ShibFields`. Once this data is successfully validated and parsed, the system instantiates an internal User model, officially granting the Student User role access to the Schedule Subsystem to begin data retrieval.

12. State & Session Management

To maintain application security and performance, user sessions are strictly managed. The system enforces a session timeout limit of **2 to 4 hours of idle time**. During an active session, tokens or session data are securely stored using **Apache Tomcat server session memory utilizing HttpOnly and Secure flagged cookies**. The overall strategy for state persistence across the application is defined as: **transient, in-memory POJO storage bound to the active user session lifecycle**.

Because the SISCalendar application acts as an intermediate data broker between the upstream SIS Gateway API and downstream client outputs, state management is intentionally designed to be ephemeral. The system strictly avoids utilizing a persistent local database for storing sensitive academic records. Instead, all retrieved schedule data - including specific course information,

meeting patterns, and exam details - is stored entirely as Plain Old Java Objects (POJOs). These POJOs reside temporarily within the Java 1.7.0_40 environment running on the underlying Apache Tomcat server. For example, after the schedule is fetched and parsed by the system, the resulting Java objects are bound directly to the user's active session using internal Struts 2 execution paths like `Session.set("courses", courses)`. This transient state strategy guarantees that once the 2-to-4-hour idle timeout threshold is reached, or the user explicitly logs out, the in-memory POJOs are destroyed, leaving no residual sensitive data warehoused on the underlying Linux host.

Securing the session identifiers utilized by the Tomcat container is paramount to the integrity of the application. To protect the authenticated connection established by the `SingleSignInHandler`, it is explicitly mandated that all session tokens and tracking cookies generated by the server must be strictly configured with the `HttpOnly` and `Secure` flags. The inclusion of the `HttpOnly` flag provides a critical mitigation against basic cross-site scripting (XSS) vulnerabilities, as it prevents client-side scripts (such as those executed via jQuery) from accessing the session token. Concurrently, the `Secure` flag guarantees that the session identifier is never transmitted over unencrypted, plaintext HTTP connections, strictly limiting token transmission to encrypted HTTPS channels. This defense-in-depth approach to session management ensures that the connection between the student's web browser or mobile device and the Apache Tomcat application server remains resilient against token interception and session hijacking attacks.

13. Data Flow & Boundaries

The SISCalendar system is strategically positioned as a central, transient integration hub that brokers real-time data between client-side user interfaces and multiple highly secure external enterprise endpoints. Because the application functions as a unified gateway merging authentication, academic records, geographic mapping, and external cloud calendars, establishing rigid and secure data flow boundaries is paramount. To guarantee absolute security and data integrity across these diverse network perimeters, it is explicitly defined that all external in-transit communication uses standard HTTPS/TLS 1.2 or higher cryptographic protocols. Furthermore, while the application architecture primarily utilizes transient memory for data handling, any sensitive cached data at rest within the underlying Linux and Tomcat server environments is strictly encrypted using standard AES algorithms to proactively prevent unauthorized local access or data extraction.

The primary upstream data ingestion boundary exists between the SISCalendar backend and the university's internal records system. Upon successful user verification via the Shibboleth Authentication System, the application's Gateway component initiates a direct, synchronous request across the network boundary to the SIS Gateway API. The SIS Gateway API responds by transmitting the student's complete academic schedule payload back across the boundary, strictly formatted as JSON, which is subsequently parsed by the internal JSONParser into the Java memory models. A secondary upstream ingestion boundary interfaces with the RIT Interactive Map API. The application consumes geographic coordinate data and building tooltips from this API to render customized visual map overlays on the client-side interface.

Conversely, the downstream data export boundaries dictate how processed schedule information leaves the SISCalendar system. The most direct export boundary is between the SISCalendar application server and the authenticated user's Web Browser or Mobile Device. In this flow, the internal CalBuilder module packages the POJO data into an ICS formatted file, which is then transmitted over HTTPS directly to the client as a standardized iCal download. Alternatively, the user can authorize a cloud-based export boundary linking the application directly to their personal Google account. In this automated workflow, the internal gCalBuilder module securely traverses the network boundary to authenticate with Google Auth Servers, programmatically pushing the parsed schedule payload directly into the Google Calendar API without any manual file handling by the user.

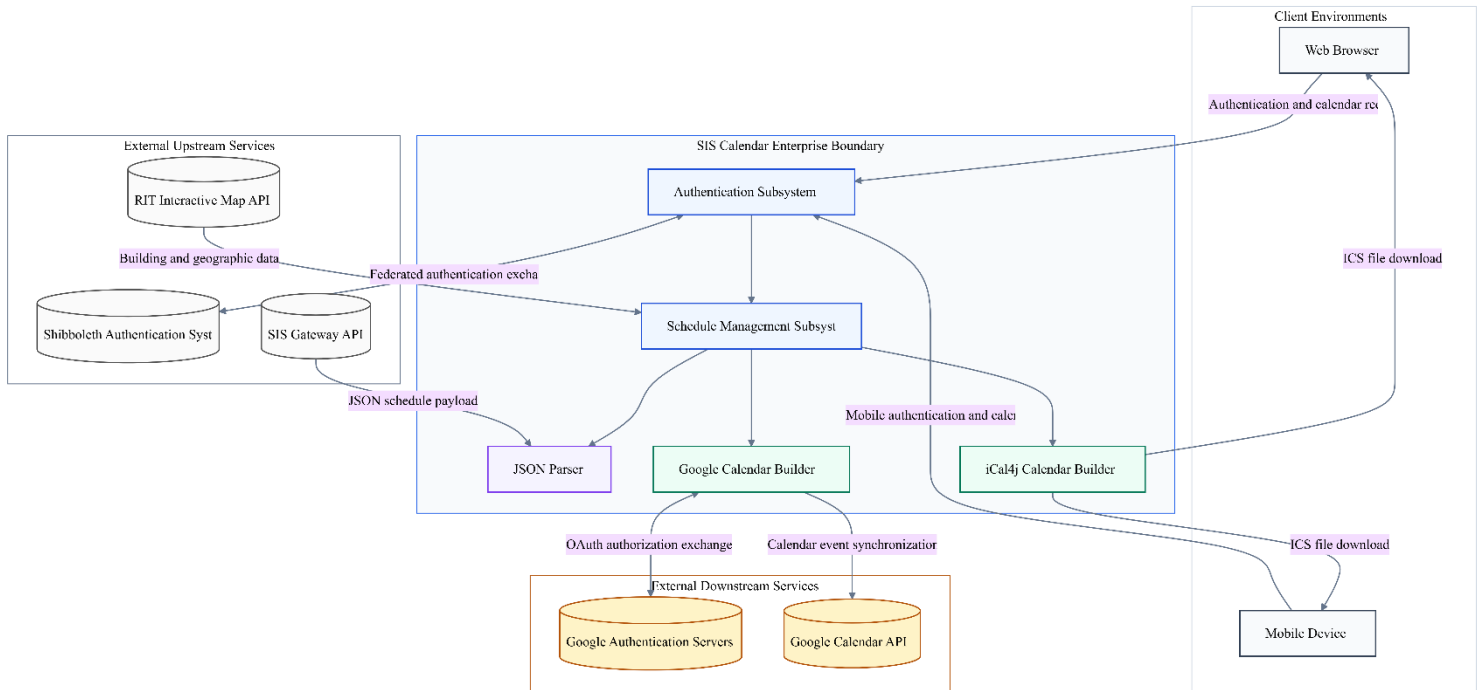


Figure 3 SIS Calendar Enterprise Architecture and Integration Flow Diagram

When data is transmitted from the **SIS Gateway API** to the **SISCalendar Schedule Subsystem**, the communication utilizes the **HTTPS/TLS 1.2+** protocol. The expected payload type for this exchange is **JSON**. To ensure security in transit, all data crossing this boundary is encrypted using **standard SSL/TLS Certificates**.

When data is transmitted from the **SISCalendar Application** to the **Google Calendar API**, the communication utilizes the **HTTPS/TLS 1.2+** protocol. The expected payload type for this exchange is **RESTful API JSON Objects**. To ensure security in transit, all data crossing this boundary is encrypted using **OAuth 2.0 authentication tokens and TLS encryption**.

When data is transmitted from the **SISCalendar CalBuilder** to the **User Web Browser / Mobile Device**, the communication utilizes the **HTTPS/TLS 1.2+** protocol. The expected payload type for this exchange is **ICS (iCal Document)**. To ensure security in transit, all data crossing this boundary is encrypted using **standard SSL/TLS Certificates**.

14. Error Handling Guidelines

The SISCalendar application architecture relies heavily on synchronous communication with several external systems, including the university's primary databases and third-party cloud services. Because the application functions as a transient middleware layer utilizing Plain Old Java Objects (POJOs) generated from external data, establishing robust error handling guidelines is critically important to maintain system stability. When a student initiates a login or a schedule fetch, the application makes direct calls to the external Shibboleth Authentication System and the SIS Gateway API. If either of these upstream dependencies experiences latency, network timeouts, or complete outages, the SISCalendar system must gracefully catch these connection exceptions. It is strictly mandated that the application must specify safe default failure states for any connection drop to the SIS Gateway API. Instead of cascading the failure and crashing the underlying Java 1.7.0_40 environment or the active user session, the system must intercept the timeout within the Struts 2 actions layer and return the user to a stable, isolated dashboard state displaying a friendly localized error message indicating that academic records are temporarily unreachable.

Furthermore, a critical component of the application's security posture is the strict management of error outputs generated by the underlying server infrastructure. Under no circumstances should raw exception data be exposed to the client interface. It is explicitly mandated that all detailed internal error messages and Apache Tomcat stack traces are completely suppressed in the production environment. Allowing Tomcat to render standard Java stack traces to the user's browser provides malicious actors with a direct blueprint of the internal application structure, library versions, and execution paths. Instead, the web.xml configuration and Struts global exception mappings must be configured to catch all unhandled exceptions and route the user to a generic, sanitized error page. Diagnostic information must be silently written to the internal log files, such as catalina.out, which reside securely on the Linux server host.

The application is designed with specific error handling guidelines to prevent information leakage and ensure stability. In the event of a critical failure, the system defaults to a **safe, isolated failure state where SIS Gateway connection drops do not terminate the underlying Tomcat server or active user session**. Within production environments, stack trace visibility is explicitly set to: **completely suppressed and sanitized, hiding all detailed internal error messages from the client interface**.

Furthermore, the system implements standardized responses for common exceptions:

- Upon encountering the **SIS_GATEWAY_TIMEOUT** error, the system will execute the following handling strategy: The Struts 2 Gateway action will intercept the connection exception, preserve the user's active authenticated session, and render a generic user-friendly warning on the JSP view indicating that academic data is currently unavailable, while silently logging the specific network failure to the catalina.out file.
- Upon encountering the **GOOGLE_API_AUTH_FAILURE** error, the system will execute the following handling strategy: The internal gCalBuilder module will abort the scheduled calendar push, prevent any partial data uploads to the Google Calendar API, and return the user to the main interface with a prompt asking them to re-authenticate their Google account credentials securely.
- Upon encountering the **SHIB_UNAUTHORIZED_ACCESS** error, the system will execute the following handling strategy: The SingleSignOnHandler will immediately invalidate any existing transient session data, destroy the internal User POJO, and redirect the client entirely out of the application back to the standard university login portal.

15. System Telemetry & Logging

Because the SISCalendar system operates entirely on a transient, in-memory data model utilizing Plain Old Java Objects (POJOs), persistent database records of user activity do not exist. Therefore, system telemetry and logging serve as the absolute critical foundation for maintaining operational awareness, auditing security events, and debugging complex network interactions. The system relies entirely on the robust logging capabilities provided by the underlying web server environment to trace the execution flow of the Struts 2 actions and monitor the health of the synchronous data fetching mechanisms. By centralizing these logs, the engineering and operations teams can rapidly isolate whether a reported failure is originating from the internal parsing logic, the external RIT Gateway, or the third-party Google authentication servers.

Operational excellence and security monitoring are supported through comprehensive system telemetry. All logs are aggregated using **the standard Apache Tomcat catalina.out file, which acts as the centralized destination for all standard output and standard error streams generated by the Java runtime**. To protect sensitive information, the system employs the following PII and sensitive data redaction strategy before logs are written: **Basic Personally**

Identifiable Information (PII) redaction is strictly mandated at the application logging layer. This ensures that highly sensitive identifiers, such as raw student authentication tokens, exact academic grades, and plain-text passwords, are proactively stripped or masked using regular expression filtering before any diagnostic event string is ever committed to the local server disk.

The engineering team utilizes specific administrative commands, such as `tail -f -n 100 ~/instances/sisCalendarCluster/node1818/logs/catalina.out`, to monitor these aggregated logs in real-time across the production node cluster. This continuous monitoring is essential for quickly identifying active threats or performance bottlenecks without interrupting the active application processes.

The following core events are captured by the logging system:

- Category: **Authentication Subsystem Events** - This encompasses: the explicit tracking of all basic Shibboleth login successes and failures. The application will immediately write a log entry when the `SingleSignOnHandler` verifies a user, or conversely, when unauthorized access attempts occur, providing a vital audit trail for potential intrusion detection.
- Category: **External API Integrity and Network Health** - This encompasses: the continuous monitoring of the application's upstream connections, explicitly logging Gateway API timeouts and connection drops. If the `JSONParser` fails to receive the expected payload due to latency, this event is cataloged to diagnose infrastructure issues.
- Category: **Schedule Subsystem Execution** - This encompasses: tracking the internal micro-interactions, such as the successful generation of a downloadable file via the `CalBuilder` or the successful execution of an automated schedule push utilizing the `gCalBuilder`.

16. External Dependencies & Libraries

The SISCalendar application architecture heavily leverages a carefully curated technology stack, relying on several critical external libraries to bridge the gap between the university's internal student information systems and the modern, mobile-friendly interface expected by students. By utilizing established, third-party frameworks like Struts 2 and dedicated external application

programming interfaces (APIs), the development team avoids the massive overhead of engineering complex web routing, calendar formatting, and cloud synchronization tools from scratch. However, introducing external code into the university's ecosystem requires strict governance. Therefore, a rigorous dependency management strategy is enforced to ensure these libraries do not introduce security risks or stability issues over the application's lifecycle.

The system relies on verified third-party components to deliver its functionality.

- The application utilizes **Apache Struts 2 framework** (Version: **2.3.15.2**). The primary purpose of this dependency is to provide: the foundational Model-View-Controller (MVC) architecture for the enterprise web application. It supplies the core "action" controller classes responsible for intercepting user requests, executing the underlying business logic, and routing the formatted POJO data directly to the JSP view layer for rendering. The strategy for updating and maintaining this library is defined as: **a standard dependency patching protocol where engineering teams regularly monitor Apache security bulletins and execute version updates through Apache ANT build scripts to proactively patch known public vulnerabilities, specifically targeting injection or cross-site scripting flaws common in MVC frameworks.**
- The application utilizes **iCal4j** (Version: **Latest Compatible Release**). The primary purpose of this dependency is to provide: a specialized Java translation engine capable of seamlessly transforming the application's internal Plain Old Java Objects (POJOs) containing parsed course and exam information into a universally standardized iCal file format. The strategy for updating and maintaining this library is defined as: **scheduled periodic audits of the open-source repository to ensure ongoing compatibility with modern desktop and mobile calendar specifications, and the prompt application of dependency updates to patch any known public vulnerabilities related to file construction or memory handling.**
- The application utilizes **google-api** (Version: **Latest Compatible Release**). The primary purpose of this dependency is to provide: the necessary cryptographic protocols and data models to interface deeply with Google's custom service ecosystem, enabling the gCalBuilder module to automatically authenticate and upload class information directly to a student's personal Google Calendar account. The strategy for updating and maintaining this library is defined as: **continuous alignment with Google's official deprecation**

schedules, utilizing automated dependency checking to swiftly adopt new API versions and patch known public vulnerabilities before legacy authentication endpoints or data transmission methods are retired by the vendor.

17. Environment Configuration

Environment configuration for the SISCalendar system is strictly decoupled from the core application source code to ensure high portability, reliable builds, and maximum security across the development, staging, and production environments. The primary method for managing this configuration and orchestrating the build lifecycle relies heavily on Apache ANT scripts. ANT serves as the foundational Java library utilized to automate the compilation of the Struts application and package it into deployable Web Application Archive (.war) files. By leveraging the precedent set by the Team Builder project, the deployment engineers utilize these specialized ANT scripts to execute complex cluster deployments, ensuring that the application can be smoothly transitioned to the target Linux host servers using command-line tools like scp and ssh.

A fundamental pillar of this configuration management strategy is the absolute prohibition of hardcoding sensitive application secrets within the application's source code repositories. The system requires secure connections to several external endpoints, notably requiring specialized authentication tokens to interface with the Google Calendar API. It is explicitly mandated that all sensitive configuration materials, including Google API keys, Shibboleth configuration parameters, and any internal system passwords, must be read dynamically from secure host environment variables at runtime. Under no circumstances are these API keys to be committed, pushed, or hardcoded within the local Git repositories or the centralized SVN (Subversion) release directories (such as trunk/dev/release/). By injecting these secrets exclusively at the runtime layer on the target Linux servers, the system prevents credential leakage and ensures that the compiled .war files remain entirely agnostic of the sensitive environments they operate within.

Configuration is managed separately from the application code to ensure portability and security across different deployment environments. The primary configuration management tool utilized is **Apache ANT scripting framework**. Sensitive values, such as API keys and database credentials, are securely injected into the runtime environment using **dynamic host operating system environment variables and isolated server-side property files**.

Specific environment configurations are handled as follows:

- For the **Production (siscalendar.rit.edu)** environment, variables are designated with the prefix **SIS_CAL_PROD_**. These configuration values are stored within: **the highly restricted environment shell profiles and protected property files located within the Linux host operating system.**
- For the **Staging / Development (siscalendardev.rit.edu)** environment, variables are designated with the prefix **SIS_CAL_STG_**. These configuration values are stored within: **the local /home/<username>/ directory profiles on the remote development Linux host.**

18. Deployment Architecture

The system is deployed on infrastructure provided by **RIT Information and Technology Services (ITS)**. The overall network topology and segmentation strategy is described as: **a standard multi-tiered enterprise architecture that strictly separates the public-facing application routing layer from the internal, highly secured academic data repositories and authentication subnets.** To secure the network perimeter, the following ingress and egress controls are enforced: **standard firewall restrictions allowing public HTTPS (port 443) ingress exclusively to the Apache Tomcat application servers, while restricting direct SSH (port 22) administrative and deployment access strictly to authenticated campus network IP addresses or verified university VPN connections.**

The deployment architecture of the SISCalendar application is heavily engineered around a robust, script-driven pipeline that pushes compiled artifacts to the remote Linux environments. To ensure operational consistency, the development team follows the established deployment precedent set by the university's Team Builder project. The core artifact of the application is a Web Application Archive (.war) file, which is programmatically constructed using Apache ANT build scripts. This automation guarantees that the Struts 2 application, JSP views, and internal Java classes are packaged correctly for the target Apache Tomcat server instances.

The physical deployment to the designated staging and development servers, specifically siscalendardev.rit.edu, is executed via secure shell protocols. The engineering team utilizes a tiered access approach. In a direct deployment scenario without utilizing the Subversion (SVN) repository, a developer first runs the ANT build script locally to generate the dist/sisCalendar.war file. This file is then securely transferred across the network boundary to the target Linux host

using the Secure Copy Protocol (SCP) via the command: `scp dist/sisCalendar.war <username>@siscalendardev.rit.edu:~/`. Following the secure transfer, the engineer establishes a remote terminal session using SSH (`ssh <username>@siscalendardev.rit.edu`). Once authenticated on the server, the deployment requires privilege escalation; the engineer executes `sudo su - webapps` to assume the necessary administrative role, copies the archive into the staging directory (`cp /home/<username>/sisCalendar.war ~/staging/sisCalendar.war`), and finally triggers the automated cluster rollout script located at `bin/cluster/chgMgmt/deploySisCalendar`.

Alternatively, the architecture supports a highly controlled, SVN-integrated deployment pathway for formal releases. In this paradigm, the compiled .war file is copied into the `trunk/dev/release/` directory and formally checked into version control using `svn ci -m "<CHECKIN MESSAGE>"`. The engineer then SSHs into `siscalendardev.rit.edu`, escalates privileges, and utilizes specific change management scripts, executing `bin/cluster/chgMgmt/getReleasedSisCalendar` followed by the standard `deploySisCalendar` command to pull the verified artifact directly from the repository into the live Tomcat cluster.

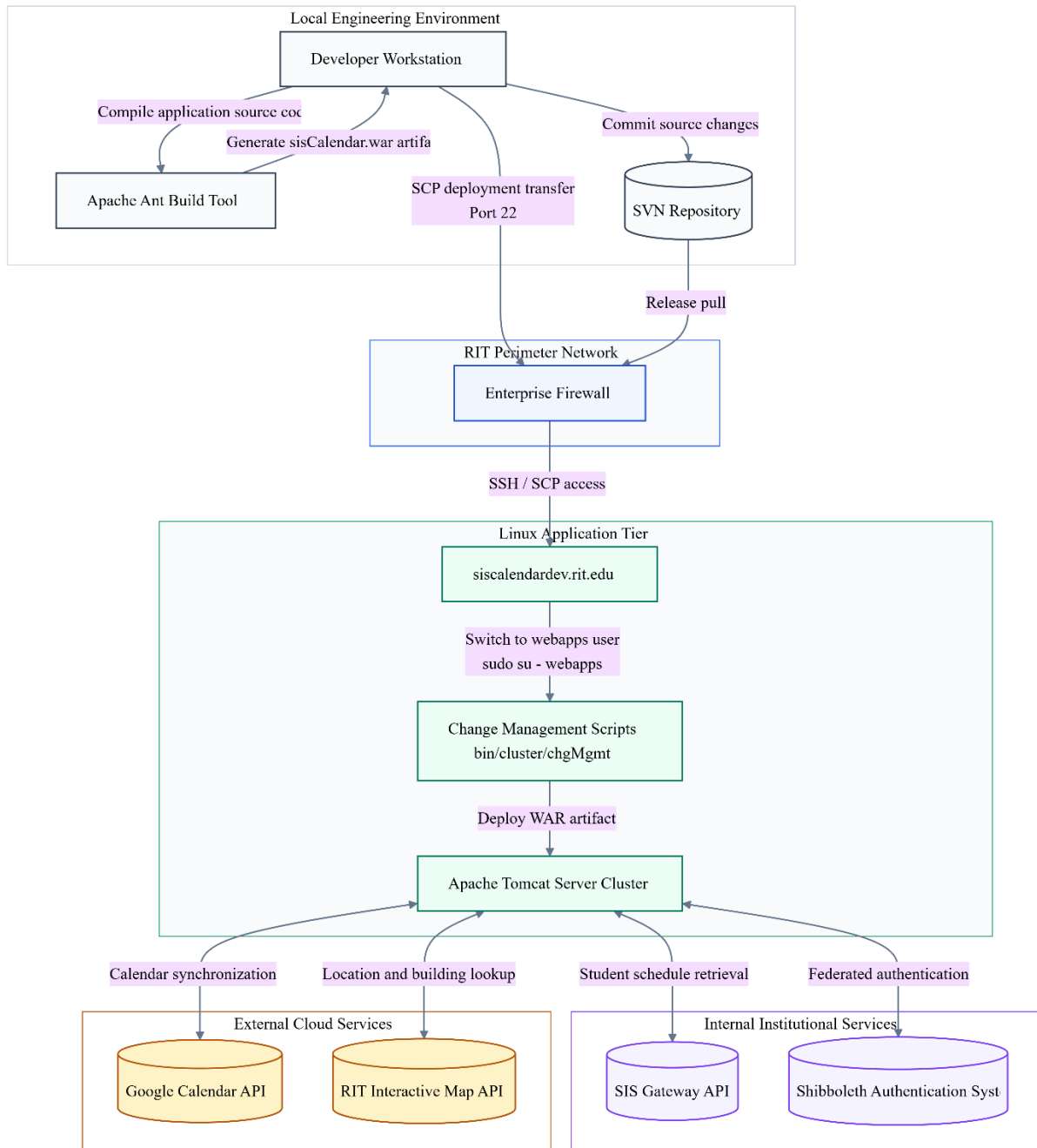


Figure 4 SIS Calendar Deployment and Runtime Integration Architecture

The deployment architecture consists of the following key nodes:

- Node: **Developer Workstation** - The localized environment running Eclipse Kepler 4.3 IDE where feature branching, raw Java development, and initial Apache ANT compilation

take place.

- Node: **siscalendardev.rit.edu Linux Server** - The primary remote Linux host operating system responsible for receiving secure file transfers (via SCP), executing deployment scripts, and maintaining the Java 1.7.0_40 environment.
- Node: **Apache Tomcat Cluster** - The dedicated web server application tier running within the Linux host that actively serves the deployed sisCalendar.war file, manages user sessions, and executes the Struts 2 Model-View-Controller logic.

19. Continuity & Recovery Operations

To ensure business continuity in the event of a disruption, recovery operations are clearly defined. The target Recovery Time Objective (RTO) is **4 hours**, and the target Recovery Point Objective (RPO) is **24 hours**.

The backup strategy mandates that backups are performed at a frequency of **daily**. All backup data is securely stored at **centralized university disaster recovery storage clusters** and is protected using **standard AES-256** encryption.

Because the SISCalendar system fundamentally operates as a transient middleware broker, the continuity and recovery operations are uniquely streamlined. The application utilizes a Plain Old Java Object (POJO) model to temporarily store course and user data entirely in memory during an active session. Consequently, there is no localized, persistent relational database within the SISCalendar environment that requires complex point-in-time restoration following a critical failure. If the Apache Tomcat server or the underlying Linux host experiences a catastrophic crash, the system will not suffer any permanent data loss regarding student academic records, as this sensitive information remains securely warehoused upstream within the university's primary SIS Gateway databases.

Therefore, recovery operations focus exclusively on rapidly provisioning the underlying server infrastructure and restoring the application's executable state. The 4-hour Recovery Time Objective (RTO) dictates the maximum acceptable downtime required for system administrators to spin up a new Linux host, deploy the Java 1.7.0_40 runtime environment, configure the necessary Shibboleth authentication parameters, and execute the ANT deployment scripts to push the latest stable sisCalendar.war file into production. The 24-hour Recovery Point Objective

(RPO) applies strictly to the daily snapshots taken of the host server's configuration files, localized shell scripts, and the aggregated catalina.out system logs. By maintaining daily encrypted backups of these structural components, the operations team ensures that the server can be rapidly rebuilt to its exact prior state without necessitating manual reconfiguration.

20. Data Lifecycle Management

Data processed by the system is subject to strict lifecycle management policies to ensure compliance and optimize storage.

While individual student schedules are completely ephemeral and purged immediately upon session termination or timeout, the system does manage specific overarching datasets to improve performance and reduce redundant API calls to external services. The application architecture specifically implements a BuildingsCacheJob and a HolidayCalendarCacheJob to handle data that remains relatively static but requires periodic refreshing to maintain accuracy.

The BuildingsCacheJob is designed to securely warehouse the topological coordinates and building metadata retrieved from the RIT Interactive Map API. Caching this geographic data is crucial to ensure that the user interface renders map overlays rapidly without forcing the system to query the external RIT map servers for every individual course a student is enrolled in. Similarly, the HolidayCalendarCacheJob aggregates and stores university-wide scheduling exceptions, such as academic breaks and recognized holidays. This overarching calendar is fundamentally necessary for the CalBuilder module to accurately construct an iCal file, ensuring that recurring meeting patterns are properly interrupted during designated university closures. Because campus layouts and academic calendars are subject to administrative changes, these cached datasets must adhere to rigid lifecycle policies to prevent the system from serving stale or deprecated information to the student body.

- Data categorized as **Cached University Building Locations (BuildingsCacheJob)** is subject to a retention period of **7 days**. Upon reaching the end of this period, the archival strategy employed is **complete omission from long-term storage, as the data is easily reproducible via the upstream API**. Once the data is no longer required for archival purposes, it is securely destroyed using the following disposal method: **automated memory garbage collection triggered by the cache refresh script, which purges the existing Java objects and initiates a fresh pull from the RIT Interactive Map API**.

- Data categorized as **University Exception and Holiday Records (HolidayCalendarCacheJob)** is subject to a retention period of **30 days (or the duration of the current academic term)**. Upon reaching the end of this period, the archival strategy employed is **complete omission from long-term storage, relying entirely on the authoritative upstream academic registry**. Once the data is no longer required for archival purposes, it is securely destroyed using the following disposal method: **automated memory garbage collection within the Java environment, flushing the outdated holiday POJOs to synchronize with the upcoming academic term's official schedule**.

21. Assumptions and Constraints

Assumptions: The successful deployment, operational stability, and continuous maintenance of the SISCalendar system are heavily predicated upon several foundational assumptions regarding the external university infrastructure, third-party cloud service availability, and the execution environment. It is assumed that the primary infrastructure provided by RIT Information and Technology Services (ITS) - specifically the Apache Tomcat web servers running on Linux distributions - will remain highly available and properly configured with the necessary Java 1.7.0_40 runtime environment.

Furthermore, the system relies on the assumption that the upstream SIS Gateway API will consistently return well-formed, structurally predictable JSON payloads containing the required course, exam, and meeting pattern properties. Any unannounced alterations to the data schema provided by the SIS Gateway will directly cause the internal JSONParser to fail. It is also assumed that the recycled Shibboleth authentication code, initially developed by the Team Builder project, will remain secure, compliant with university policies, and fully compatible with future iterations of the RIT identity provider systems. Finally, for users utilizing the automated cloud synchronization features, it is assumed they possess active, valid Google Accounts and that Google will maintain backward compatibility for their core calendar authentication APIs during the system's operational lifecycle.

- The underlying RIT ITS infrastructure, including the Linux host servers, Apache Tomcat instances, and enterprise firewalls, will be maintained, patched, and kept highly available by the university operations team.

- The SIS Gateway Web Services API will reliably execute requests and return standard, consistently formatted JSON data payloads that match the expected internal schema of the SISCalendar JSONParser module.
- The established Shibboleth Authentication System provided by the Team Builder project implementation will continue to securely validate credentials without requiring extensive architectural overhauls by the SIS.io development team.
- The external RIT Interactive Map API will remain accessible and provide accurate, up-to-date topographical coordinates and building data to render customized map overlays correctly.
- Students utilizing the gCalBuilder automated synchronization functionality possess valid, active Google profiles and consent to the OAuth token exchanges required to write data to their personal Google Calendars.

Constraints: The constraints represent the rigid boundaries, non-negotiable architectural limitations, and operational bottlenecks imposed upon the SISCalendar system by the university environment and the chosen technology stack. The most critical constraint governing the entire application is its absolute reliance on the availability of the SIS Gateway Web Services. Because the system operates entirely on a transient in-memory data model and does not persistently warehouse academic records locally, the application is fundamentally rendered inoperable if the network connection to the SIS Gateway drops or the external API experiences downtime. The application cannot serve cached offline schedules to users if the initial login and data fetch sequence cannot be completed.

Additionally, the system is strictly constrained by a read-only boundary regarding upstream university data. The SISCalendar application has zero authorization or functional capability to write, modify, drop, or alter any student registration data within the primary academic databases. Development constraints also heavily influenced the architectural design; due to strict project schedule limitations, the team was constrained to reuse the Team Builder project's Shibboleth authentication implementation rather than engineering a custom security protocol. Finally, the deployment environment imposes rigid versioning constraints: the application must be compiled for and strictly operate within a Java 1.7.0_40 environment, utilizing the Apache Struts 2.3.15.2 framework, limiting the adoption of newer language features or MVC libraries.

- **SIS Gateway Availability Dependency:** The application's core functionality is entirely dependent on the real-time availability of the SIS Gateway API; any upstream network latency or API downtime will directly result in a total denial of service for schedule fetching.
- **Read-Only Data Boundary:** The application operates under a strict read-only mandate; it is physically and programmatically constrained from writing or modifying any student registration records within the university's primary databases.
- **Transient Memory Architecture:** The system is constrained to a transient data model, relying exclusively on Plain Old Java Objects (POJOs) stored in volatile server memory during active sessions, preventing the long-term local storage or retrieval of historical user schedules.
- **Strict Versioning Requirements:** The development and production environments are strictly locked to the Java 1.7.0_40 runtime and the Apache Struts 2.3.15.2 MVC framework, constraining the team from utilizing newer software libraries or language syntaxes.
- **Authentication Code Reuse:** Due to restrictive project timelines and university security mandates, the development architecture is constrained to utilizing the pre-existing Shibbolethsecurity implementations derived from the Team Builder project.

22. Glossary

The following glossary defines the critical technical terminology, external libraries, internal components, and architectural frameworks utilized throughout the SISCalendar Technical Specification Document. Understanding these terms is essential for developers, system administrators, and project stakeholders to accurately comprehend the data flow, security boundaries, and operational dependencies of the application.

Term	Definition
POJO	Plain Old Java Object. This represents the foundational, lightweight data structure utilized by the SISCalendar system to transiently store all operational data in active memory. Course details, exam information, and user sessions are instantiated as POJOs, allowing them to be easily passed to and manipulated by the Struts Actions layer without the heavy overhead of a traditional database framework.

Shibboleth	The robust, university-wide authentication and authorization system utilized by RIT. SISCalendar relies on this external security system to rigorously verify a user's identity and credentials before granting them access to the application's core Schedule Subsystem.
iCal4j	A dedicated, third-party Java library integrated directly into the SISCalendar Business Services layer. It is explicitly responsible for transforming the internal POJO data structures into a universally standardized iCal format, ultimately enabling the system to create downloadable .ics calendar files for the end-user.
ANT	Apache ANT is a highly reliable Java-based build tool and scripting library utilized extensively by the development team. It automates the complex compilation and deployment processes, seamlessly converting the raw Struts application source code into deployable Web Application Archive (.war) files for the Apache Tomcat servers.
Struts 2	An enterprise-grade Model-View-Controller (MVC) framework published by Apache. It serves as the architectural backbone of the SISCalendar application, providing out-of-the-box "action" controller classes that handle logic execution and route processed data directly to the view layers.
JSP	Java Server Pages. A high-level server-side technology that provides an abstraction of Java servlets. It is utilized in the application's View layer to cache, dynamically alter, and render the final HTML user interface sent to the client's web browser.
SIS Gateway API	The primary external, read-only Web Services interface utilized by the SISCalendar backend to securely retrieve the raw, comprehensive academic course information associated with the authenticated student's university account.
gCalBuilder	An internal Java module created by the SIS.io team that utilizes the external google-api library. Its specific purpose is to securely authenticate with remote Google servers and automatically push a student's parsed class information directly into their personal Google Calendar account.

23. Approval

This Technical Specification Document represents the finalized architectural blueprint, data flow definitions, and operational constraints for the SISCalendar application. Signatures below indicate that the system design has been reviewed and formally accepted by the core development team and project stakeholders.

Approved By: Zach Masiello, Ethan Mick, Michael Caputo, Shawn Thompson (Organization: SIS.io)

Approval Date: 04/12/14